

CS2 Analytics

End-to-End Data Analytics, Machine Learning & Prediction App

A professional Counter-Strike 2 analytics project that turns raw match data into a relational database, SQL findings, predictive models, a user-facing round-win prediction app, and an interactive Tableau dashboard.

16,527

rounds analyzed

111,715

kill events

71.84%

best model accuracy

Core question

What factors are associated with winning a professional CS2 round, and how accurately can the winner be predicted using information available around the opening kill?

[PostgreSQL](#) • [SQL](#) • [Python](#) • [pandas](#) • [scikit-learn](#) • [Streamlit](#) • [Tableau](#) • [Git/GitHub](#)

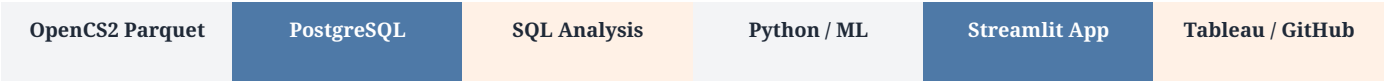
[OpenCS2 Dataset](#)

Project Overview

From raw event data to an interactive prediction product and dashboard

ARCHITECTURE

End-to-End Workflow



The project starts with structured professional CS2 match data, models it relationally in PostgreSQL, uses SQL for exploratory and event-level analysis, and then passes curated features into Python for classification. The best model is exposed through a lightweight Streamlit app, while Tableau presents the major analytical findings.

SCOPE

What the Project Demonstrates

Area	Implementation
Database engineering	Relational schema, composite keys, foreign keys, data loading and indexing
SQL analytics	Aggregations, joins, CASE logic, CTEs, window functions and LEAD()
Machine learning	Preprocessing pipeline, one-hot encoding, logistic regression and random forest
Application layer	Streamlit form for entering round context and returning CT/T win probabilities
Visualization	Interactive Tableau dashboard with KPIs and comparative charts
Delivery	Git/GitHub project structure, commits, README and public dashboard link

Data & Database Design

Structured gameplay data modeled for repeatable analysis

DATA

Dataset Scale

794	16,527	111,715	168,294
match / map rows	rounds	kills	player-round rows

The source data is stored in Parquet files. Python and pandas are used first to inspect fields, identify keys and validate row counts before the data is loaded into PostgreSQL.

POSTGRESQL

Relational Schema

Table	Primary key	Relationship / purpose
matches	(match_id, map_id)	One record per map within a professional match
rounds	(match_id, map_id, round)	Round outcome and round-level attributes; FK to matches
kills	kill_id	Individual kill events; FK to rounds
round_player	(match_id, map_id, round, player_slot)	Player performance by round; FK to rounds

Why the schema matters

The relational structure allows analysis to move from match-level context to round outcomes and then down to event-level kills or player-level statistics without flattening the entire project into one file.

PIPELINE

Load & Validation

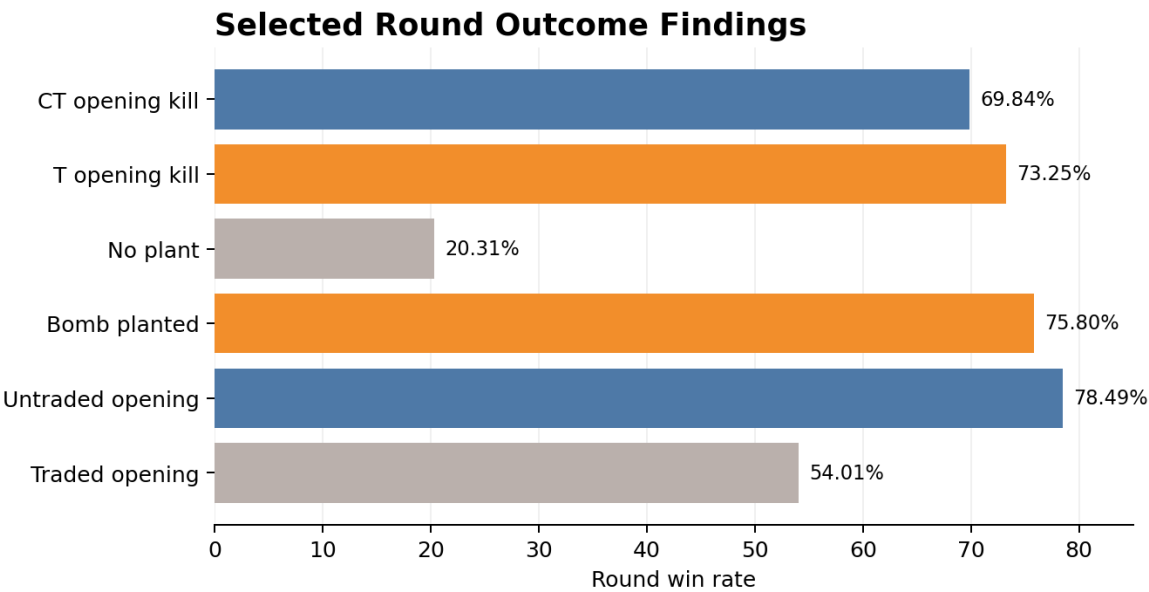
- Python reads the Parquet files and loads records into PostgreSQL in dependency order.
- Primary and foreign keys enforce relationships between matches, rounds, kills and player-round records.
- SQL COUNT(*) checks confirm the database matches the source row counts.
- PostgreSQL becomes the primary analytical source after the load is complete.

SQL Analysis

Exploring the game events most associated with round outcomes

ANALYSIS

Selected Findings



Opening-kill timing itself changed win rate only slightly, while the side that secured the opening kill mattered much more. Bomb plants were strongly associated with T-side success, and a quick trade reduced most of the opening-kill advantage.

IMPLEMENTATION

SQL Techniques Used

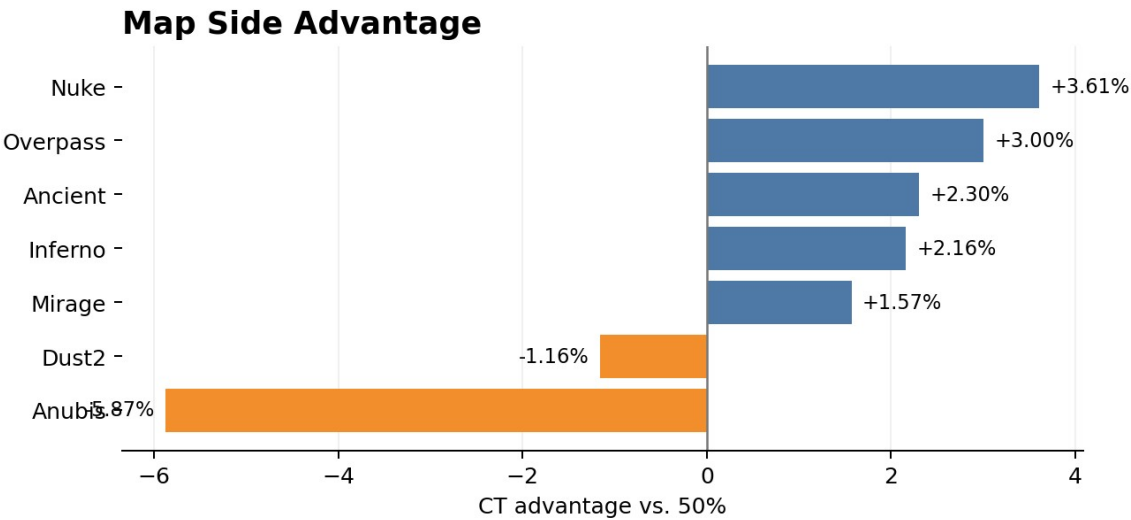
Technique	How it was used
Aggregations / GROUP BY	Win rates by side, map, timing bucket and gameplay state
CASE expressions	Binary indicators and grouped analytical categories
Joins	Combining round outcomes with event-level kill information
CTEs	Breaking multi-step event logic into readable stages
Window functions / LEAD()	Identifying the next kill after the opening kill to detect trades
Conditional aggregation	Comparing outcomes under different round states

Round Context & Trade Analysis

Moving beyond single-variable summaries

MAPS

Map Side Advantage



Nuke is the most CT-leaning map in this sample, while Anubis is the most T-leaning. The dashboard expresses this as deviation from a 50% CT win rate, making side bias easy to compare.

EVENT LOGIC

Opening-Kill Trades

Key result

When the opening kill is not traded within five seconds, the opening side wins about 78.5% of rounds. When it is quickly traded, that falls to about 54.0% - close to removing the initial advantage.

A follow-up analysis grouped trade response times into 0-2, 2-5, 5-10 and 10+ second buckets. Once the opposing side gets the next kill, the exact response time shows much less relationship with the final outcome than the existence of the trade itself.

OPTIMIZATION

Performance Check

A representative kill-event query was profiled with EXPLAIN ANALYZE. Adding a composite index changed the plan from a sequential scan to an index scan and reduced execution time from 12.868 ms to 0.085 ms. This optimization is included as a supporting database-performance exercise rather than the central focus of the project.

Machine Learning

Predicting CT vs. T round winners from early-round context

MODELING

Prediction Question

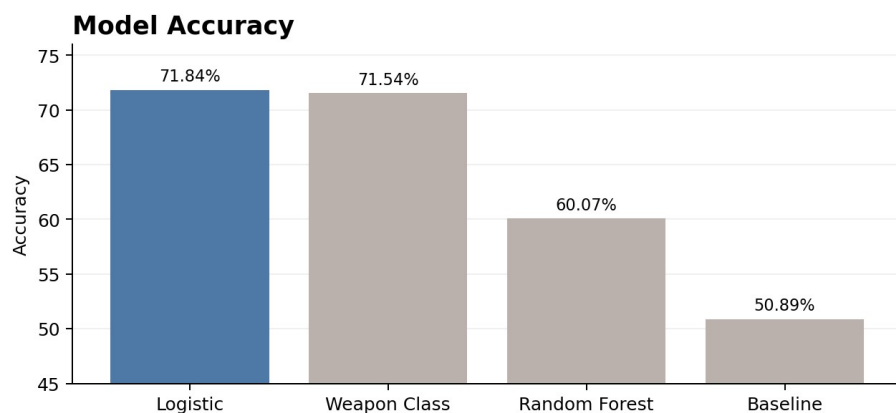
Target

Given the map, opening-kill side, opening-kill timing and opening weapon, predict whether CT or T wins the round.

SCIKIT-LEARN

Modeling Pipeline

- SQL results are pulled into pandas from PostgreSQL.
- Invalid or missing opening-kill rows are removed.
- Categorical fields such as map, opening-kill side and opening weapon are one-hot encoded.
- A ColumnTransformer/Pipeline keeps preprocessing consistent at training and prediction.
- Models are evaluated on a held-out test set against a majority-class baseline.
- Late-round variables are excluded to avoid data leakage.



Logistic Regression performs best at 71.84% accuracy versus a 50.89% baseline. Grouping weapons into broader classes produces nearly the same accuracy (71.54%) while reducing rare-weapon noise and making the feature effects easier to explain.

Interpretation

Opening-kill side is the strongest signal in the model. Map adds context, while weapon or weapon class contributes a smaller amount of predictive information.

Round-Win Prediction App

Turning the model into a simple user-facing tool

PRODUCT LAYER

Streamlit Application

The project includes a Streamlit app that packages the trained round-win model into an interactive scenario tool. Instead of reading model output from a terminal, a user can enter the early-round context and immediately receive a prediction.

USER INPUT	MODEL	OUTPUT
Map Opening-kill side Opening-kill time Opening weapon	Saved scikit-learn Logistic Regression pipeline (model.pkl)	Predicted winner CT / T + class probabilities

INTEGRATION

Why the App Matters

- Demonstrates that the model can be used outside the analysis notebook or terminal.
- Reuses the same trained preprocessing/model pipeline, reducing mismatch between training and inference.
- Makes the project easier to demonstrate in a portfolio or interview because the prediction can be tested interactively.
- Connects the analytical work to a small product experience rather than stopping at model evaluation.

Example use

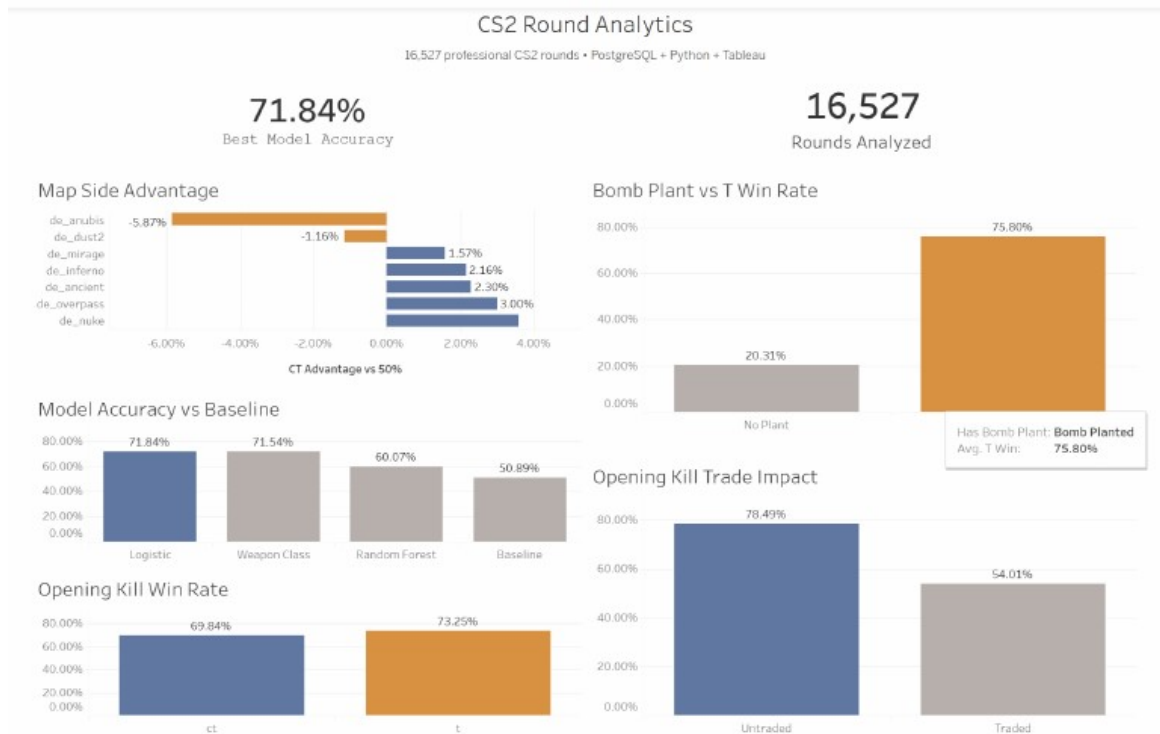
A user selects a map, indicates which side got the opening kill, enters the opening-kill timing and weapon, and the app returns the predicted CT/T winner with probabilities.

Tableau Dashboard

A compact presentation layer for the strongest findings

VISUALIZATION

CS2 Round Analytics



[View the interactive Tableau dashboard](#)

The dashboard combines headline KPIs with map-side advantage, bomb-plant impact, opening-kill win rate, trade impact and model comparison. CT-related marks use blue, T-related marks use orange, and neutral comparisons use gray for a consistent visual language.

PRESENTATION

Dashboard Design Choices

- KPI cards surface 16,527 rounds analyzed and the 71.84% best-model accuracy.
- Map advantage is centered at 50% so CT-leaning and T-leaning maps diverge around a neutral baseline.
- The best model is highlighted while comparison models remain muted.
- Technical field labels are removed where the chart title and data labels already explain the measure.

Project Delivery

Organized code, version control and documentation



GITHUB

Repository Structure

Area	Purpose / examples
data/	Source or local dataset files
sql/	schema.sql, analysis.sql, optimization.sql
python/	inspect_data.py, load_data.py, analysis.py, export_tableau.py
app.py	Streamlit prediction interface
tableau/	Workbook and exported analytical CSVs
images/	Dashboard screenshot for the README
README.md	Project overview, findings, model results and dashboard link

GIT

Development Workflow

Git is used throughout the project rather than only at the end. Major stages - database setup, SQL analysis, query optimization, Python modeling, Tableau work and documentation - are committed separately so the repository shows how the project developed over time.

TOOLS

Technology Stack

Layer	Tools
Data / database	OpenCS2 Parquet, PostgreSQL, SQL
Analysis	Python, pandas, SQLAlchemy
Machine learning	scikit-learn, Logistic Regression, Random Forest
Application	Streamlit, serialized scikit-learn model
Visualization	Tableau Public
Version control	Git, GitHub

Key Outcomes

What the completed project shows

RESULTS

Analytical Findings

- The side securing the opening kill wins roughly 70-73% of rounds.
- An untraded opening kill corresponds to about a 78.5% round win rate; a quick trade drops that to about 54.0%.
- Bomb plants are strongly associated with T-side success: about 75.8% T win rate after a plant versus 20.3% without one.
- Map context matters: Anubis is T-leaning in this sample, while Nuke is CT-leaning.
- Logistic Regression is the strongest tested model at 71.84% accuracy, with opening-kill side as the dominant predictor.

PORTFOLIO VALUE

Skills Demonstrated

Skill area	Evidence in the project
Database design	Composite primary keys, foreign keys and relational modeling
SQL	Joins, CTEs, CASE, window functions, LEAD(), aggregation
Performance	EXPLAIN ANALYZE and composite indexing
Python / pandas	Database retrieval, cleaning, feature preparation and exports
Machine learning	Pipelines, encoding, model comparison, metrics and interpretation
Application development	Interactive Streamlit prediction tool using the saved model
Data visualization	Tableau KPIs, comparative charts and interactive dashboard
Software workflow	Git, GitHub, project structure and README documentation

Final result

An end-to-end analytics project that connects raw professional CS2 data to a relational database, SQL analysis, predictive modeling, an interactive prediction app, a Tableau dashboard and a documented GitHub portfolio repository.